



PCIe Native2axistream Reference Design

上海安路信息科技股份有限公司

APUG032 (v1.1) 2023 年 12 月



目 录

目 录	I
1 概要	1
2 设计规格	1
2.1 接口属性	1
2.2 系统框图	1
2.3 资源消耗	2
3 文件目录说明	2
4 设计接口与功能介绍	3
4.1 顶层模块接口	4
4.2 顶层模块时序	6
4.3 子模块功能	6
4.4 配置空间额外配置	7
5 仿真	7
5.1 测试激励介绍	8
5.2 仿真波形分析	8
6 上板调试	10
6.1 软硬件环境准备	10
6.2 PCIe 硬核配置	10
6.3 编译与测试	12
版本信息	13



免责声明	13
------------	----



1 概要

AXI-Stream 接口是广泛应用于 FPGA 设计的接口标准，而安路科技 PH1 系列 FPGA 的 PCIe 控制器使用的是自定义的 Native 接口。为兼容 AXI-Stream 接口，本方案提供了将安路 PCIe 的 Native 接口转换为 AXI-Stream 总线接口的方案，便于用户直接替换已有 AXI-Stream 的设计。

2 设计规格

本设计 PCI Express 符合 PCI Express Base Specification Revision 5.0, Version 1.0 标准协议，所提供的 AXI-Stream 接口与 Xilinx 的 7 系列 PCIe AXI Stream 数据接口兼容，包括发送接口和接收接口。

2.1 接口属性

native2axistream 接口目前仅支持 RC 端寄存器读写和 EP 端 DMA 读写，其它如 System (SYS) interface、Configuration (CFG) interface 等暂未支持，详细的 TLP 支持类型如下图 2-1 所示：

表 2-1 设计支持的操作

操作类型	是否支持	3DW 包头	4DW 包头	数据位宽	有效负载范围	说明
RC-PIO-MWR	支持	支持	支持	64bit	1DW	仅支持 DW 读写
RC-PIO-MRD	支持	支持	支持	64bit	1DW	
EP-DMA-MWR	支持	支持	支持	64bit	1DW~MPS/4	仅支持 DW 对齐
EP-DMA-MRD	支持	支持	支持	64bit	1DW~MRR/4	
RC-CFGW	硬核支持	\	\	\	\	
RC-CFGR	硬核支持	\	\	\	\	

由于逻辑部分收敛时序的限制，当前设计仅支持 GEN1、GEN2 和适配的 X1 和 X2，要想跑到 PCIe 硬核支持的 GEN3，逻辑代码需要做更多的优化。

注释：操作类型中第一个字段 RC 或者 EP 表示包文的发起方，中间 PIO, DMA 表示操作的方式，MRW, MRD 表示 TLP 包文的格式，CFGW, CFGR 表示配置包文，配置包文由 PCIe 硬核回复，本模块目前不响应。

2.2 系统框图

系统框图如下图 2-2 所示，pcie_ep_core 把接收到的 TLP 传递给 native2axistream_ip 模块，如果 TLP 为 MWR 或 MRD 类型，由 trgt1_rx 模块处理此类型包，并提取 TLP 的 header 和 data 送到 radm2axistream 模块；如果 TLP 为 CPLD 类型，则转交给 bypass_rx 模块处理，同样送出 header 和 data 到 radm2axistream 模块。radm2axistream 模块首先分别存储上述类型包，然后根据 rx_arb 提供的优先级顺序从缓存中读出数据组 m_axis_rx 包送出到用户模块 native2axistream_app。



native2axistream_app 模块接收 m_axis_rx 传过来的包，根据 TLP 类型存储数据或解析参数，如果收到了 DMA 写参数和写起始信号则组 DMA 写包，如果收到 MRD 请求则返回 CPLD 包。

native2axistream_app 模块返回的 s_axis_tx 包被送到 axistream2xali 模块，axistream2xali 模块根据 TLP 类型分别解析出不同的 header 和 data 送到 client0_tx 模块，client0_tx 存储各类型的数据并根据 tx_arb 提供的优先级从缓存中读出数据组 client0-tlp 包送出到 pcie_ep_core。

注释：框图中的 p (posted) 对应 MWR，np (non-posted) 对应 MRD，cpld 也就是 CPLD 包。

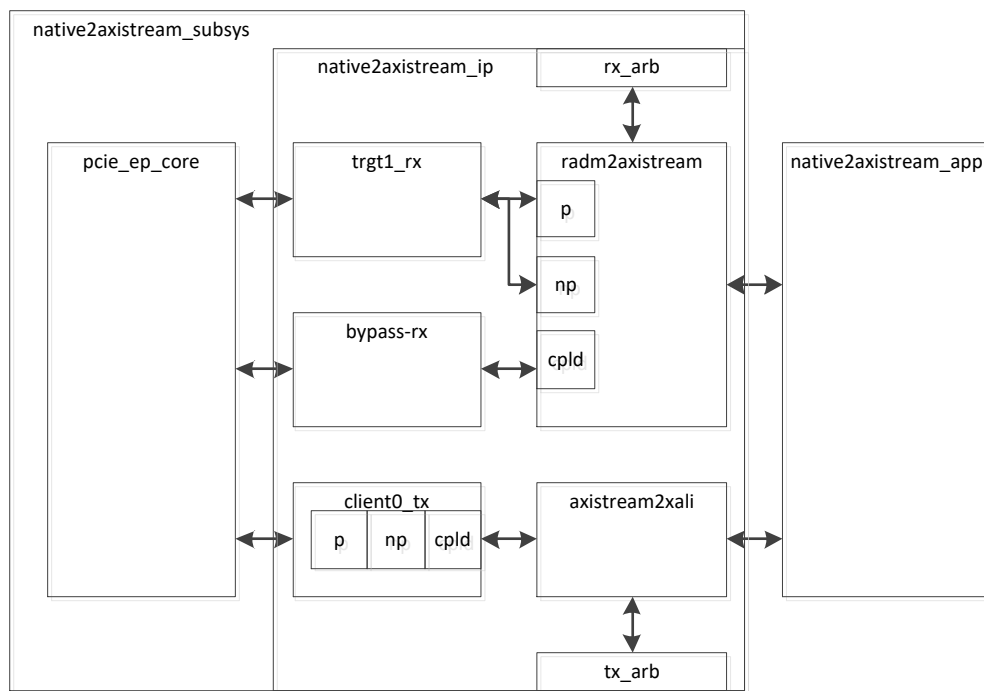


图 2-2 系统框图

2.3 资源消耗

如上图 2-2 所示，设计中已考虑了 DMA 读，并包含了 DMA 读模块，但 EP-DMA-MRD 需要下一个版本才能支持。整个工程的资源消耗如下图所示（来自于 TD5.6.4 编译报告）。

表 2-3 Native2axistream_subsys_exam 消耗资源

名称	WNS	WHS	LUT6	REG	ERAM	DSP	PLL	GCLK	PCIe
数值	0.195	0.003	3259	3891	13	1	1	3	1

3 文件目录说明

文件目录内包含四部分：仿真工程 tb_native2axistream，上板测试工程 native2axistream_prj、Linux 驱动和参考文档，目录各文件的简要介绍如下表 3-1 所示：

表 3-1 文件目录说明



目录	文件	文件说明
\impl\ep_prj	native2axistream_prj.al	td 工程文件
\impl\ep_prj\constraint	case_ep.sdc	工程时序约束文件
	case_ep.adc	pin 约束及 pll 位置约束文件
\impl\rc_prj		暂未放 rc 的工程
\src\native2axistream_app	\src*.v	app 所用源文件
\src\native2axistream_ip	\def*.vh	native2axistream_ip 中用到的宏定义
	\ip*.v	native2axistream_ip 中用到 ip 文件
	\ip_sim*.v	native2axistream_ip 中用到 ip 仿真文件
	\src*.v	native2axistream_ip 中用到源文件
\src\top	*.v	native2axistream_ip 顶层文件
\sw	readme.txt	驱动使用说明
	\anlogic_pcie_drv*.c *.h	驱动源文件
	\anlogic_pcie_app*.c *.h	驱动 app 文件
\tb	\bfm*.v	rc 端接口仿真模型文件
	\data\case**.dat	tb 数据, 包括 golden 和 sim
	\def*.vh	tb 中用到的宏定义文件
	\ip_sim*.v	tb 中用到的 ip 仿真文件
	\qsim *.do *.f	tb 中的 modelsim 仿真文件
	\top*.v	tb 的顶层文件
\doc	*.docx	参考设计文档及调试记录
\bit	*.bit	准备好的下载文件

4 设计接口与功能介绍

本设计完成 PCIe 到 AXI-Stream 接口的转换, 顶层结构如下图 4-1 所示。下文首先介绍顶层模块接口和时序, 随后简要介绍各子模块的功能, 最后补充配置空间的额外配置。



图 4-1 native2axistream_subsys 顶层结构图

4.1 顶层模块接口

顶层模块 native2axistream_subsys 合并了 pcie_ep_core 和 native2axistream_ip 两个子模块，使得对外接口只剩下 axistream 接口和 PCIe 相关的部分输入输出接口，它们的功能描述如下表 4-2 所述，表中的 i 代表输入端口，o 代表输出端口。

表 4-2 native2axistream_subsys 接口介绍

信号名称	方向	描述
app_auxclk_in	i	为PCIe硬核提供常在的辅助时钟。
pcie_rst_n	i	全局复位（复位后配置信息会丢失）
refclk_p	i	PCIe硬核参考时钟100MHz，差分。
refclk_n	i	
txp	o	PCIe硬核输出Serdes信号，差分。
txn	o	
rxp	i	PCIe硬核输入Serdes信号，差分。
rxn	i	
user_lnk_up	o	Data link层链接指示信号，高有效。
user_clk	o	PCIe硬核输出core_clk经pll处理后的输出时钟。
user_resetn	o	PCIe硬核输出core_rst_n经pll输出时钟处理后的输出时钟
core_clk_led	o	PCIe正常工作的指示信号，与PCIe的Generation有关
s_axis_tx_tready_o		native2axistream_subsys模块准备好接收数据，s_axis_tx_tvalid和s_axis_tx_tready的断言标志着一个数据的成功传输。



s_axis_tx_tdata_i[63:0]	i	传输的数据包
s_axis_tx_tkeep_i[7:0]	i	传输数据字节使能, s_axis_tx_tkeep[0] ==1b, s_axis_tx_tdata[7:0] 有效; s_axis_tx_tkeep[7] ==0b, s_axis_tx_tdata[63:56]无效; 当s_axis_tx_tlast没到时 s_axis_tx_tkeep_i必须全部为0xff。
s_axis_tx_tuser_i[3:0]	i	s_axis_tx_tuser[3]对应t_src_dsc, 发送源中断; s_axis_tx_tuser[2]对应tx_str, 传输流模式; ts_axis_tx_tuser[1]对应tx_err_fwd, 向前传输错误; ts_axis_tx_tuser[1]对应tx_ecrc_gen, 接收ECRC错误。
s_axis_tx_tlast_i	i	传输包的结束, s_axis_tx_tvalid断言时有效。
s_axis_tx_tvalid_i	i	传输源准备好, 表示app已经准备好数据在s_axis_tx_tdata上。
m_axis_rx_tdata_o[63:0]	o	接收的数据。
m_axis_rx_tkeep_o	o	接收数据字节使能, 与s_axis_tx_tkeep_i类似。
m_axis_rx_tlast_o	o	接收包的结束, m_axis_rx_tvalid断言时有效。
m_axis_rx_tvalid_o	o	接收源准备好, 表示native2axistream_subsys模块已经准备好数据在s_axis_tx_tdata。
m_axis_rx_tready_i	i	app准备好接收数据, m_axis_tx_tvalid和m_axis_tx_tready的断言标志着一个数据的成功传输。
m_axis_rx_tuser_o[21:0]	o	m_axis_rx_tuser[1]对应rx_err_fwd, 向前接收错误; m_axis_rx_tuser[0]对应rx_ecrc_err, 接收ECRC错误; m_axis_rx_tuser[16:15]保留; m_axis_rx_tuser[9:2]对应rx_bar_hit[7:0], 表示当前Receive事务对应的BAR空间。从包的开始断言到m_axis_rx_tlast结束。 m_axis_rx_tuser[2]对应rx_bar_hit[0], 指示 BAR0; m_axis_rx_tuser[3]对应rx_bar_hit[1], 指示 BAR1; m_axis_rx_tuser[4]对应rx_bar_hit[2], 指示 BAR2; m_axis_rx_tuser[5]对应rx_bar_hit[3], 指示 BAR3; m_axis_rx_tuser[6]对应rx_bar_hit[4], 指示 BAR4; m_axis_rx_tuser[7]对应rx_bar_hit[5], 指示 BAR5;



		m_axis_rx_tuser[8]对应rx_bar_hit[6]，指示 Expansion ROM Address；如果64bit地址，对应的两个rx_bar_hit bit位都有效；m_axis_rx_tuser[9]保留；
--	--	--

4.2 顶层模块时序

native2axistream_subsys 总线接口分成两类:m_axis_rx 和 s_axis_tx。m_axis_rx 用于转接 MWR、MRD 和 CPLD 包并传递到 app 模块，本转模块支持的 AXI 格式跟 xilinx 的 PG054 保持一致。

本设计支持 X 厂手册上提到的 ‘Source Throttling on the Transmit Datapath’ , ‘Destination Throttling of the Transmit Datapath’ , ‘ Destination Throttling on the Transmit Interface’ , ‘ Streaming Mode on the Transmit Interface’ 。不支持 X 厂提到的 ‘ Source Driven Transaction Discontinue on the Transmit Interface’ , ‘ Discarding of Transaction by Destination of Transmit Interface’ , ‘ Packet Data Poisoning on the Transmit Interface’ , 支持时序如下表:

格式类型	是否支持	对应时序图
TLP 3-DW Header without Payload	☑	Figure 3-3:
TLP with 4-DW Header without Payload	☑	Figure 3-4
TLP with 3-DW Header with Payload	☑	Figure 3-5
TLP with 4-DW Header with Payload	☑	Figure 3-6
Source Throttling on the Transmit Interface	☑	Figure 3-8
Destination Throttling of the Transmit Datapath	☑	Figure 3-36
Destination Throttling on the Transmit Interface	☑	Figure 3-9:
Streaming Mode on the Transmit Interface	☑	Figure 3-13
Source Driven Transaction Discontinue on the Transmit Interface	☒	Figure 3-37
Discarding of Transaction by Destination of Transmit Interface	☒	Figure 3-11
Packet Data Poisoning on the Transmit Interface	☒	Figure 3-12

4.3 子模块功能

顶层文件 native2axistream_subsys 下的 native2axistream_ip 有七个子模块，下面分别简要介绍各模块功能。首先是 trgt1_rx.v 模块，它一方面接收 MWR 和 MRD 包，并把它转换成统一的 header 和 data 包时序；另一方面存储 MRD 的 trgt_lookup_id，并在回 CPLD 包时同时送出。

bypass_rx.v 模块接收 EP 发起 DMA MRD 请求返回的 CPLD 包，并把它转换成同 trgt1_rx 输出一样的 header 和 data 包时序。

rx_arb.v 模块存储上述三种类型的包顺序，并在检测到存储空间非空时，读出包类型，并以此产生对应包类型的区域。当前只支持顺序读写的仲裁方式，后续根据需要可以很容易的支持其它仲裁方式。

radm2axistream.v 模块首先存储各类型包数据，然后根据仲裁后的包优先级从缓存中读出数据，最



后根据数据包类型和大小组成 `s_axis_rx` 包送到 `app` 模块。

`axistream2xali.v` 模块接收三种类型的 `s_axis_tx` 包，分别为：CPLD 包、DMA-MWR 包、DMA-MRD 包，然后根据包类型把它合并成统一的 `header` 和 `data` 包时序。

`client0_tx.v` 模块有两方面的功能，一方面根据包类型把上面的 `header` 和 `data` 存入不同的缓存，另一方面根据仲裁模块给出的顺序从缓存中读出数据产生 PCIe 时序要求的 TLP。

`tx_arb.v` 模块同 `rx_arb.v` 实现方式相似，首先存储三种包类型的顺序，然后再缓存非空时读出包类型，最后产生各种类型包的时序区域。

4.4 配置空间额外配置

PCIe 硬核默认把 BAR0 和 BAR1 映射到 ELBI 接口，用户根据需要可通过 DBI 总线修改配置空间寄存器，让所有 BAR 空间全部映射到 TRGT1 接口，为此需要在 PCIe 启动正常后配置以下寄存器。

表 4-11 BAR 空间全部映射到 TRGT1 需修改的寄存器

寄存器名	地址	默认值	配置值	描述
TRGT_MAP_CTRL_OFF	0x81c	0x1f7e	16'h1fff	禁止 BAR0 寄存器对应 ELBI 访问接口，而是 TRGT1 接口

`app` 端 DMA 负载允许的最大值也可通过 DBI 总线读取获得，对应的 DBI 总线地址如下表 4-12 所示。

表 4-12 链路两端协商后确定的包负载寄存器

寄存器名	地址	默认值	配置值	描述
DEVICE_CONTROL_DEVICE_STATU S	0x07 8	0x10211 0	/	bit[7:5] 对应 max_payload_size, bit[14:12] 对应 max_read_request_size

5 仿真

当完成设计代码后，下一步即开始仿真，仿真由测试仿真模型和测试激励构成，本设计搭建的测试仿真例程如下框图 5-1 所示，数据流向由以下几部分组成：

1. `tb_native2axistream` 模块内的 `tb_execute` 段代码配置 RC 和 EP 的配置空间，完成之后根据需要开始配置 EP-DMA-MWR 或 EP-DMA-MRD 寄存器，并发起 DMA 起始信号；`pcie_rc_case0` 的 `t1p_access_rc` 模块把收到的 `mem_wr` 和 `mem_rd` 命令封装成标准的 PCIe 包送到 `pcie_rc_core` 模块；`pcie_rc_core` 把 `t1p` 总线转换成 `serdes` 总线。

2. `pcie_ep_core` 接收 `serdes` 总线数据并把它转换成标准的 PCIe 包；`native2axistream_ip` 模块根据包类型分别转换成不同的 `m_axis_rx` 包；`native2axistream_app` 模块根据收到的包类型或存储、或返回 CPLD 包、或发起 DMA 读写包；这些 `s_axis_tx` 包经过 `native2axistream_ip` 成 PCIe 的标准包送到 `pcie_ep_core`。

3. `pcie_rc_core` 接收到 `pcie_ep_core` 转换过来的 `serdes` 总线，并把它转换成标准的 PCIe 包，



如果是 bypass 包被 tlp_access_rc 接收, 否则被 pcie_rx_bfm 接收; 如果接收的是 EP-DMA-MRD 包, 将会由 pcie_tx_bfm 产生完成包并发出。

4. pcie_ep_core 收到完成包再交由 native2axistream_ip 转换成 m_axis_rx 包送 native2axistream_app 模块使用; 至此, 一个完整的读写循环完成。

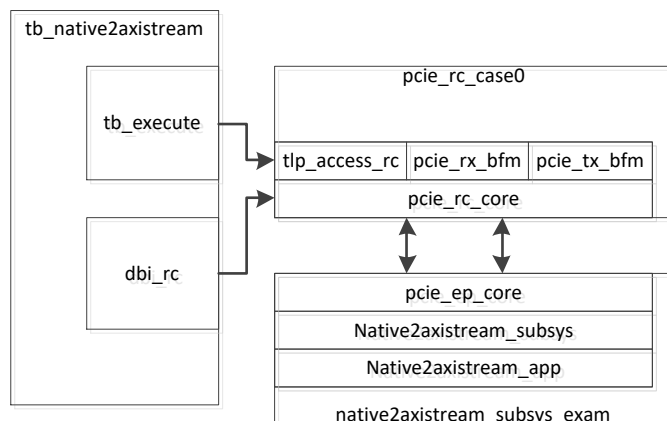


图 5-1 测试激励实现框图

5.1 测试激励介绍

上文叙述了测试例程中数据流的完整流向, 下面将简要介绍 tb_native2axistream 测试激励中的核心代码 tb_execute 部分。它由两段构成: DBI 总线读写各自配置空间, MWR 和 MRD 读写 DMA 各寄存器。DMA 寄存器配置与 Xilinx 的 xapp1052.pdf 完全一致, 此处不做介绍。DBI 读写配置空间的几个关键寄存器如下表 5-2 所示:

表 5-2 关键寄存器配置

寄存器名	地址	默认值	配置值	描述
BAR0_REG	0x10	0x4	bit[2:0]=4'd0	配置成非预读, 32bit, MEM空间
BAR1_REG	0x14	0x0		
BAR2_REG	0x18	0x0		
BAR3_REG	0x1C	0x0		
STATUS_COMMAND_REG/RC	0x4	0x110000	bit[2:0]=3'b100	启用总线主控
STATUS_COMMAND_REG/EP	0x4	0x110000	bit[2:0]=3'b111	启用总线主控, 使能MEM和IO空间
MEM_LIMIT_MEM_BASE_REG	0x20	0x0	bit[31:0]=32'h80002000	内存限制和内存基址寄存器
DEVICE_CONTROL_DEVICE_STATUS	0x8	0x102110	bit[8:5]=4'b1100	这个寄存器控制PCI Express设备的具体参数, 并提供关于PCI Express设备(函数)的具体参数的信息。

5.2 仿真波形分析

本设计 native2axistream_subsys 内主要用到了五类包格式, 下面结合 Modelsim 仿真波形分析这些总线。首先是 Trgt1 波形, 如下图 5-3 所示, 包含 MWR-32bit 和 MRD-32bit (4DW Header 与此类似, 此处不做介绍), 64bit 的 DataPath 中的低 32 位有效。

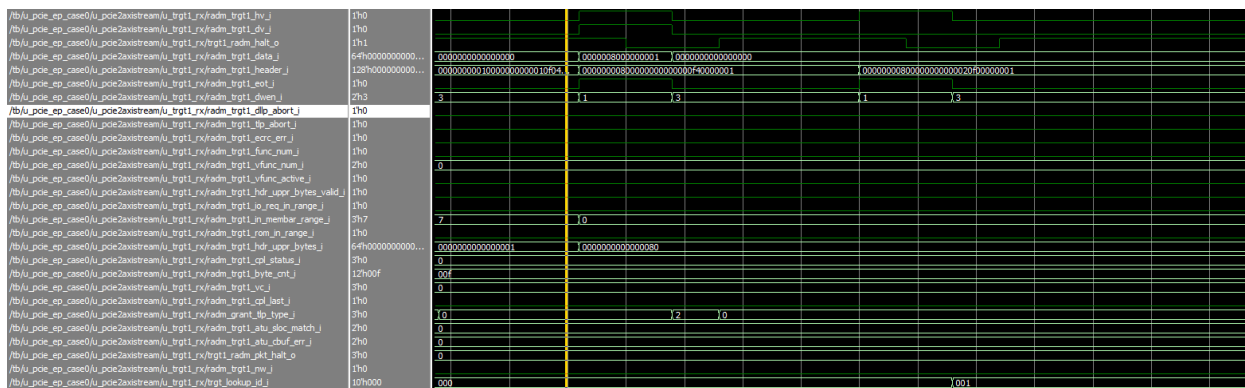


图 5-3 MWR 和 MRD 映射到 Trgt1 口的波形

由于 CPLD 映射到 Bypass 接口的波形尚未调通，此处不做介绍。接着 m_axis_rx 总线的波形，如下图 5-4 所示，它与 MWR 和 MRD 相对应。

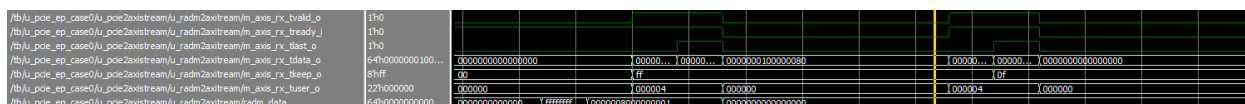


图 5-4 Trgt1 接口对应 m_axis_rx 的波形

app 模块送出的包有三种类型的包：MWR、MRD 和 CPLD，下图 5-5、5-6 分别对应 MWR 和 CPLD 包的时序。MWR 负载 1DW 与图 5-4 类似，此处不做介绍，CPLD 目前只有负载只有 1DW，4DW 包头同样不做介绍。

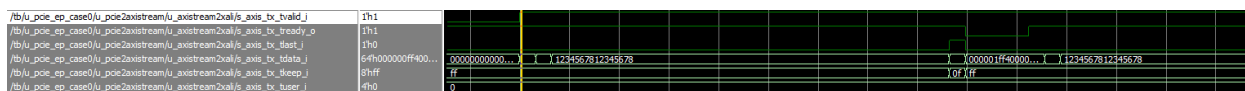


图 5-5 EP-DMA-MWR 对应 s_axis_tx 的波形

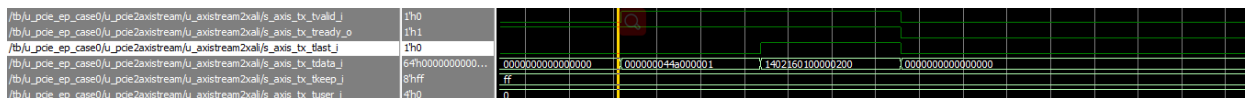


图 5-6 CPLD 对应 s_axis_tx 的波形

native2axistream_subsys 模块把收到的 MRD、MWR 和 CPLD 包转换成 TLP 包，对应的仿真时序如下图 5-7、5-8 所示，同上面一样此处没有贴 4DW Header 的波形，因为与此类似只是地址变成 64bit。

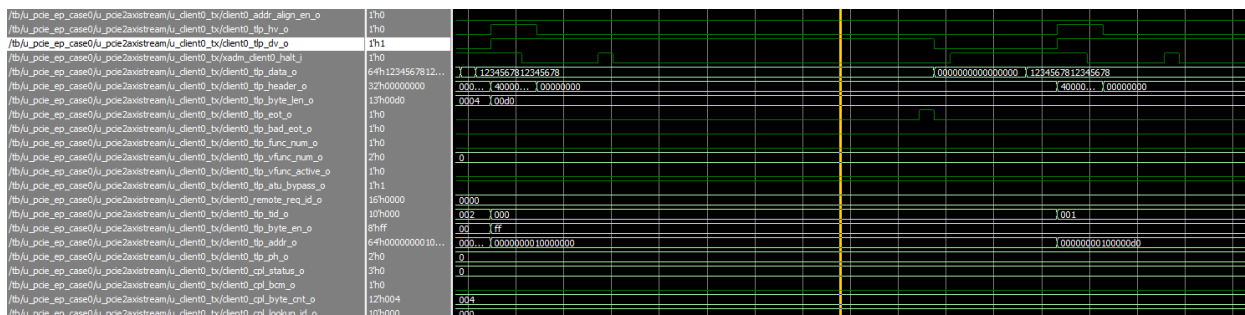


图 5-7 MWR 对应的 Client0-TLP 波形图

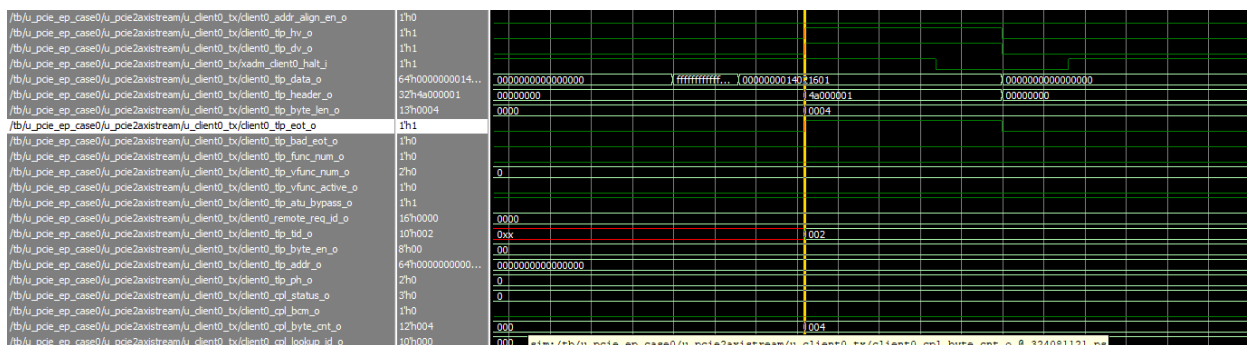


图 5-8 CPLD-32bit 对应的 Client0-TLP 波形图

6 上板调试

上板测试之前，首先准备好软硬件环境，然后生成硬核配置文件，最后合并工程开始上板调试，搭建好的软硬件测试平台如下图 6-1 所示，下面分别详细介绍这三个过程。

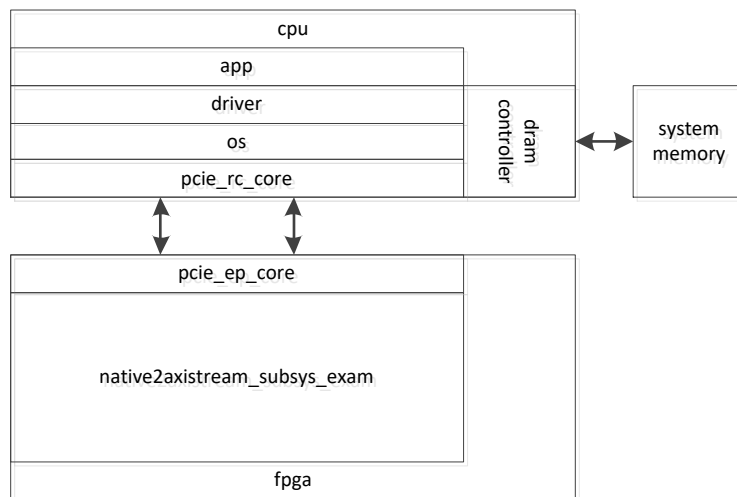


图 6-1 软硬件测试平台框架

6.1 软硬件环境准备

硬件选择公司的 DEMO 板，板载 C382/C383 无需焊接，即 100MHz 参考时钟来自 PCIe 接口。软件选择的是 Ubuntu 20.4 系统，驱动见工程目录下 ./sw/xbmd。

6.2 PCIe 硬核配置

软硬件准备好后，开始 PCIe 硬核配置，下面将截取 UI 中几项重要的配置，如下图所示。



Component: pcie_phy

PCIe Basic	Capability 1	Capability 2	ID and Class	BAR	Port Control
Device Type: PCI Express Endpoint					
PCIe Generation: Gen2					
Link Width: X2					
Link Location: X0Y2					
Reference Clock Frequency: 100Mhz					
PCIe Application Mode: Advanced					

图 6-2 PCIe Basic 配置

Component: pcie_phy

PCIe Basic	Capability 1	Capability 2	ID and Class	BAR	Port Control
Physical Functions					
☒ Physical Function 0 ☐ Physical Function 1					
PFO MAX Payload Size: 2048 Bytes PF1 MAX Payload Size					
☒ Extended Tag Field					

图 6-3 PCIe Capability1 配置

Component: pcie_phy

PCIe Basic	Capability 1	Capability 2	ID and Class	BAR	Port Control
PFO					
PF1					
PFO-VFs					
PF1-VFs					
BAR 0 Setting					
☒ BAR 0 Type: Memory ☒ 64 bit ☐ Prefetchable					
Size: 2^7 Bytes					
Value: ffffffff804 (Hex)					
BAR 2 Setting					
☒ BAR 2 Type: Memory ☐ 64 bit ☐ Prefetchable					
Size: 2^7 Bytes					
Value: ffffffff80 (Hex)					
BAR 4 Setting					
☒ BAR 4 Type: Memory ☐ 64 bit ☐ Prefetchable					
Size: 2^7 Bytes					
Value: ffffffff80 (Hex)					

图 6-4 PCIe BAR 配置



Component: pcie_phy

PCIe Basic Capability 1 Capability 2 ID and Class BAR Port Control

TX port Control

☒ XALIO

☒ XALI1

RX port Control

☒ TRGT1 (Posted/Non Posted TLP)

☒ BYPASS (Completion TLP)

Interrupt

☒ MSI

☐ MSI-X

☐ Legacy INT

☐ Vendor-Defined Message Signals

SII Interfaces

☐ AER Control Signals

☐ LTR Message Generation Signals

☐ Power Management Signals

☐ FLR (Function Level Reset) Signals

☐ ROM Validation Signals

☐ Debug Signals

☐ RAS D.E.S Related Signals

☐ Configuration Info Signals

☐ CRI Bus

☒ ELBI Signals

☒ DBI Signals

图 6-5 PCIe Port Control 配置

图 6-3 中的 PCIe Capability1 界面 PF0 MAX Payload Size 选择了 2048byte，实测链路协商都在 512byte 以内，为了减小设计中的 buffer 大小，MAX Payload Size 被限定在 512byte 以内。图 6-4 中默认只展示了 BAR0、BAR2、BAR4 的设置，如果有需要设置其它 BAR 空间，可通过 DBI 总线实现；另一方面由于 Legacy PCI 现在很少使用，所以本设计的 BAR 只支持配置成 Memory，如有 IO 类型的需要可在第一页配置成 Legacy PCI。图 6-5 默认打开了一些 PCIe 总线，用户可根据需要使能其它总线，注意打开的总线输入信号必须赋值，否则可能出现不能识别 PCIe 设备。

6.3 编译与测试

当上述步骤都完成后，就可以开始合并工程编译上板测试。编译工程最好采用最新发布的 TD 软件，本设计采用的是 TD5.6.4。编译完成 Timing 没问题，下载 pcie_brup_norm.bit 文件；然后重启电脑，进入 Ubuntu 系统，此时观察 Demo 板有一个灯在闪烁（闪烁频率与 PCIe Generation 有关），表示 PCIe 的 core_clk 起来了并且设备工作正常；最后按照 ./xbmd/执行说明.txt 的介绍开始测试，测试正常会打印 “...test OK”。



版本信息

日期	版本	修订记录
2023.06.30	1.0	文件名统一，仿真与上板工程框架同其它版本统一
2023.12.18	1.1	格式统一，调整图表内部格式与字体。

版权所有©2024 上海安路信息科技股份有限公司

未经本公司书面许可，任何单位和个人都不得擅自摘抄、复制、翻译本文档内容的部分或全部，并不得以任何形式传播。

免责声明

本文档并未授予任何知识产权的许可，并未以明示或暗示，或以禁止发言或其他方式授予任何知识产权许可；本文档仅为向用户提供使用器件的参考，协助用户正确地使用安路科技产品之用，其著作权归安路科技所有；本文档所展示的任何产品信息均不构成安路科技对所涉产品或服务作出任何明示或默示的声明或保证。

安路科技将不定期地对本文档进行更新、修订。用户如需获取最新版本的文档，可通过安路科技的官方网站（网址为：<https://www.anlogic.com>）自行查询下载，也可联系安路科技的销售人员咨询获取。